Chapitre 6:

Les tableaux

Objectifs du chapitre

- A la fin de ce chapitre, vous serez capables de :
  - Comprendre l'intérêt des tableaux en programmation.
  - Déclarer, initialiser et manipuler des tableaux à une dimension.
  - Parcourir un tableau à l'aide de boucles.
  - Résoudre des problèmes classiques sur les tableaux.
  - Déclarer et manipuler des tableaux à deux dimensions (matrices).
  - Utiliser les tableaux pour organiser et traiter des données.

Pourquoi utiliser des tableaux ?

- Supposons que l'on souhaite écrire un programme qui lit les notes de cinq étudiants et calcule leur moyenne.
- Une première solution consiste à déclarer cinq variables :

```
float n1, n2, n3, n4, n5;
```

- Puis à lire les cinq notes :

```
scanf("%f", &n1);
scanf("%f", &n2);
scanf("%f", &n3);
scanf("%f", &n4);
scanf("%f", &n5);
```

Pourquoi utiliser des tableaux ?

- Pour calculer la moyenne :

```
float moyenne;
moyenne = (n1 + n2 + n3 + n4 + n5) / 5;
printf("Moyenne = %.2f\n", moyenne);
```

- Cette solution fonctionne, mais elle présente plusieurs limites.

Les limites des variables individuelles

- Que se passe-t-il si l'on doit gérer les notes de 100 étudiants ou de 1 000 étudiants ?
- L'utilisation de variables individuelles présente plusieurs inconvénients :
  - Il faut déclarer un grand nombre de variables.
  - Le programme devient long et répétitif.
  - Il est difficile de traiter toutes les valeurs avec une seule boucle.
  - Les opérations sur l'ensemble des données deviennent compliquées.
- **Question** : comment regrouper toutes les notes sous un seul nom, tout en conservant la possibilité d'accéder à chaque note individuellement ?

La solution : Les tableaux

- Un tableau permet de regrouper plusieurs valeurs de même type sous un seul nom.
- Au lieu de déclarer cinq variables indépendantes :

```
float n1, n2, n3, n4, n5;
```

- On peut déclarer un seul tableau :

```
float notes[5];
```

- Ce tableau peut contenir cinq notes.
- On accède à chaque note grâce à son indice :

```
notes[0]
...
notes[4]
```

- **Avantage essentiel** : un tableau permet de stocker et de traiter un ensemble de valeurs avec des boucles.

Représentation d'un tableau

- Considérons le tableau suivant :

```
int T[5] = {12, 15, 9, 14, 10};
```

- En C, les éléments d'un tableau sont stockés dans des emplacements mémoire contigus. Le nom **`T`**, dans la plupart des expressions, représente l'adresse du premier élément du tableau.
- Représentation du tableau en mémoire



- Adresses hexadécimales illustratives, avec: **`sizeof(int)`** = 4 octets.

Représentation d'un tableau



- **Explication**
  - Chaque case contient un élément de type int.
  - Chaque élément occupe ici 4 octets.
  - Les adresses augmentent donc de 4 octets d'un élément au suivant.
  - L'espace mémoire réservé aux cinq éléments est contigu et occupe 5 * 4 = 20 octets.

Relation entre les indices et les adresses

- L'adresse du premier élément est :

  `&T[0]`

- Le nom **`T`** est converti en un pointeur vers le premier élément. On a donc :

  `T == &T[0]`

- L'adresse d'un élément d'indice **`i`** se calcule ainsi :

  `&T[i] = &T[0] + i * sizeof(int)`

- Cette formule décrit le calcul en octets des adresses. En C, l'expression **`&T[0] + i`** effectue directement l'arithmétique des pointeurs et désigne l'adresse de **`T[i]`**, lorsque l'indice est valide.

Définition d'un tableau

- Un tableau est une structure de données qui permet de regrouper un nombre déterminé d'éléments de même type, accessibles à l'aide d'un nom commun et d'un ou plusieurs indices.
- Les principales caractéristiques d'un tableau en C sont :
  - Homogénéité : tous les éléments ont le même type.
  - Taille déterminée : le tableau possède une capacité définie lors de sa déclaration.
  - Accès par indice : chaque élément est identifié par sa position.
  - Indices à partir de zéro : le premier élément possède l'indice 0.
- Exemple :

```
float notes[30];
```

Ce tableau peut stocker 30 valeurs de type **`float`**.

Les tableaux à une dimension

Déclaration d'un tableau

- **Syntaxe générale**

```
type nom_tableau[taille];
```

- **`type`** : type des éléments (int, float, char, etc.).
- **`nom_tableau`** : nom choisi pour le tableau.
- **`taille`** : nombre d'éléments que le tableau peut contenir.
- **Exemples**

```
int ages[10];
float notes[30];
char lettres[26];
double prix[100];
```

- Chaque déclaration réserve l'espace nécessaire pour stocker les éléments du tableau.

Comprendre la taille d'un tableau

- Considérons la déclaration :

```
int T[6];
```

- Le tableau **`T`** peut contenir **six** entiers.
- Les indices valides sont : **`0, 1, 2, 3, 4, 5`**
- L'indice maximal est donc égal à : **`taille - 1`**
- Exemples :
  - Pour **`int A[4]`**, les indices vont de 0 à 3.
  - Pour **`float B[10]`**, les indices vont de 0 à 9.
  - Pour **`char C[20]`**, les indices vont de 0 à 19.

- **Attention** : la taille représente le nombre d'éléments, et non le dernier indice.

Initialisation complète d'un tableau

- L'initialisation consiste à attribuer des valeurs aux éléments dès la déclaration.
- **Syntaxe**

```
type nom_tableau[taille] = {valeurs};
```

- **Exemple**

```
int T[5] = {10, 20, 30, 40, 50};
```

- Le tableau obtenu est :

0	1	2	3	4
10	20	30	40	50

- Chaque valeur est affectée à une case dans l'ordre de la liste d'initialisation.

Initialisation partielle

- On peut initialiser seulement quelques éléments du tableau lors de sa déclaration.

```
int T[5] = {10, 20};
```

- Les éléments non précisés sont initialisés à zéro.
- Le tableau obtenu est :

0	1	2	3	4
10	20	0	0	0

- On peut également initialiser tous les éléments à zéro :

```
int T[5] = {0};
```

- Dans ce cas, toutes les cases contiennent la valeur 0.

Déduire la taille lors de l'initialisation

- En C, on peut laisser le compilateur déterminer la taille du tableau à partir du nombre de valeurs fournies.

```
int T[] = {5, 8, 12, 15};
```

- Le compilateur déduit que le tableau possède **quatre** éléments.
- Il est équivalent à :

```
int T[4] = {5, 8, 12, 15};
```

- Cette forme est pratique lorsque toutes les valeurs sont connues au moment de la déclaration.

Accéder aux éléments d'un tableau

- Pour accéder à un élément, on utilise le nom du tableau suivi de son indice entre crochets.
- **Exemple**

```
int T[5] = {10, 20, 30, 40, 50};
printf("%d\n", T[0]);
printf("%d\n", T[2]);
```

- Résultat :

```
10
30
```

- L'expression **`T[i]`** désigne l'élément d'indice **`i`**.
- Un élément peut être utilisé dans une expression, affiché ou modifié.

Modifier les éléments d'un tableau

- On peut modifier la valeur d'une case en utilisant une affectation.

```
int T[5] = {10, 20, 30, 40, 50};
T[1] = 25;
T[3] = 100;
```

- Le tableau devient : **10 25 30 100 50**

- On peut également utiliser les valeurs des autres éléments:

```
T[2] = T[0] + T[1];
```

- Après cette instruction, **T[2]** contient **35**.
- Le tableau devient : **10 25 35 100 50**

Utiliser une variable comme indice

- L'indice d'un tableau peut être une variable entière ou une expression entière.

```
int T[5] = {2, 4, 6, 8, 10};
int i = 2;
printf("%d\n", T[i]);
printf("%d\n", T[i + 1]);
```

- Résultat :

```
6
8
```

- **Explication** : `T[i]` désigne `T[2]`, puisque `i = 2` et `T[i+1]` désigne `T[3]`.
- L'indice calculé doit toujours rester dans les limites du tableau.

Les erreurs d'indices

- Pour un tableau de taille **`n`**, les indices valides vont de **`0`** à **`n-1`**.
- Considérons : **`int T[5];`**
- Les accès suivants sont valides :

```
T[0] = 10;
T[4] = 20;
```

- Les accès suivants sont incorrects :

```
T[5] = 30;
T[-1] = 40;
```

- En C, les accès hors limites ne sont pas automatiquement vérifiés. Ils provoquent un comportement indéfini, susceptible de produire des résultats imprévisibles.
- **Règle à retenir** : toujours vérifier les limites des indices.

Parcourir un tableau

Pourquoi utiliser une boucle ?

- Un tableau contient généralement plusieurs éléments qui doivent subir le même traitement.
- Par exemple, pour afficher les éléments de :

```
int T[5] = {12, 8, 15, 6, 10};
```

- Sans boucle, il faudrait écrire cinq instructions **`printf`**.
- Avec une boucle, on peut écrire :

```
for (int i = 0; i < 5; i++) {
    printf("%d ", T[i]);
}
```

- Résultat :

```
12 8 15 6 10
```

- La boucle permet d'effectuer le même traitement sur chaque élément du tableau.

Comprendre le parcours avec for

```
for (int i = 0; i < 5; i++) {
    printf("%d ", T[i]);
}
```

- La boucle fonctionne de la manière suivante :
  1. Initialiser **`i`** à **`0`**.
  2. Vérifier si **`i < 5`**.
  3. Traiter l'élément **`T[i]`** (ici affichage par exemple).
  4. Incrémenter **`i`**.
  5. Recommencer jusqu'à ce que la condition soit fausse.
- La boucle traite successivement :

  `T[0]` → **`T[1]`** → **`T[2]`** → **`T[3]`** → **`T[4]`**
- Lorsque **`i`** devient égal à **`5`**, la boucle s'arrête.
- **Attention** : écrire **`i <= 5`** provoquerait un accès à **`T[5]`**, qui est hors limites.

Lire les éléments d'un tableau

- Le programme suivant permet de lire cinq entiers au clavier et de les afficher.

```
#include <stdio.h>
void main() {
    int T[5];
    int i;
    for (i = 0; i < 5; i++) {
        printf("Donner T[%d] : ", i);
        scanf("%d", &T[i]);
    }
    printf("Elements du tableau :\n");
    for (i = 0; i < 5; i++) {
        printf("%d ", T[i]);
    }
}
```

Lire les éléments d'un tableau

- La première boucle lit les éléments.
- La deuxième boucle affiche les éléments saisis.
- L'expression **`&T[i]`** indique à **`scanf`** l'adresse de la case où enregistrer la valeur.

Pourquoi utiliser &T[i] avec scanf ?

- Considérons :

```
scanf("%d", &T[i]);
```

- **`T[i]`** représente la valeur contenue dans la case.
- **`&T[i]`** représente l'adresse mémoire de cette case.
- La fonction **`scanf`** a besoin de l'adresse de la variable pour pouvoir y enregistrer la valeur saisie.
- Pour afficher la valeur, on utilise simplement :

```
printf("%d", T[i]);
```

- Il ne faut donc pas confondre la lecture d'une valeur et son affichage.

Utiliser une constante pour la taille

- Pour éviter de répéter la taille du tableau dans plusieurs instructions, on peut utiliser une constante symbolique.

```
#include <stdio.h>
#define N 5
void main() {
    int T[N];
    int i;
    for (i = 0; i < N; i++) {
        printf("Donner T[%d] : ", i);
        scanf("%d", &T[i]);
    }
    for (i = 0; i < N; i++) {
        printf("%d ", T[i]);
    }
}
```

Utiliser une constante pour la taille

- L'utilisation de N facilite la modification du programme.
- Si l'on souhaite gérer dix éléments au lieu de cinq, il suffit de modifier la constante.
- Le tableau reste néanmoins de taille fixe pendant l'exécution dans cet exemple.

Taille du tableau et nombre de valeurs utilisées

- Il faut distinguer la capacité d'un tableau du nombre de valeurs effectivement utilisées.
- Exemple :

```
int T[100];
int n = 5;
```

- Le tableau peut contenir jusqu'à 100 entiers, mais on peut décider de n'utiliser que les cinq premières cases.
- Les boucles de traitement doivent alors parcourir les indices de **0** à **n - 1**.

```
for (int i = 0; i < n; i++) {
    printf("%d ", T[i]);
}
```

- **A retenir** : la capacité du tableau ne correspond pas nécessairement au nombre d'éléments utilisés.

Déterminer la taille d'un tableau avec `sizeof`

- L'opérateur **`sizeof`** permet de connaître la taille occupée par un objet en mémoire, en octets.
- Pour un tableau déclaré dans le même bloc :

```
int T[10];
```

- On peut calculer son nombre d'éléments avec :

```
int n = sizeof(T) / sizeof(T[0]);
```

- **`sizeof(T)`** donne la taille totale du tableau en octets.
- **`sizeof(T[0])`** donne la taille d'un élément en octets.
- La division donne le nombre d'éléments.
- Dans cet exemple, **`n`** vaut **`10`**.

Utiliser un tableau dans une fonction

- **Exemple** : écrire une fonction qui calcule la somme des éléments d'un tableau.

```
int somme(int T[], int n) {
    int s = 0, i;
    for (i = 0; i < n; i++)
        s += T[i];
    return s;
}
```

- La fonction reçoit deux paramètres :
  - T : permet d'accéder aux éléments du tableau.
  - n : indique le nombre d'éléments à traiter.
- La taille doit être transmise séparément, car le paramètre T[] ne permet pas à la fonction de retrouver directement le nombre d'éléments du tableau d'origine.

Traitements classiques sur les tableaux

Calculer la somme des éléments

- **Principe**
  - Initialiser une variable somme à zéro.
  - Parcourir le tableau.
  - Ajouter chaque élément à la somme.
- **Programme**

```
int T[6] = {12, 5, 18, 7, 10, 8};
int somme = 0, i;
for (i = 0; i < 6; i++) {
    somme = somme + T[i]; // somme += T[i];
}
printf("Somme = %d\n", somme);
```

- Résultat :

```
Somme = 60
```

Calculer la moyenne

- La moyenne est égale à la somme des éléments divisée par leur nombre.

$$\text{Moyenne} = \frac{\text{Somme des éléments}}{\text{Nombre d'éléments}}$$

- **Programme**

```
int T[6] = {12, 5, 18, 7, 10, 8};
int somme = 0, i;
float moyenne;
for (i = 0; i < 6; i++) {
    somme += T[i];
}
moyenne = (float) somme / 6;
printf("Moyenne = %.2f\n", moyenne);
```

- Résultat :

```
Moyenne = 10.00
```

Rechercher le minimum

- Pour trouver le minimum, on commence par supposer que le premier élément est le plus petit.
- On compare ensuite chaque élément restant avec le minimum courant.
- **Programme**

```
int T[6] = {12, 5, 18, 7, 10, 8};
int i, min = T[0];
for (i = 1; i < 6; i++) {
    if (T[i] < min) {
        min = T[i];
    }
}
printf("Minimum = %d\n", min);
```

On commence à l'indice 1, car T[0] a déjà servi à initialiser min.
Cette méthode suppose que le tableau contient au moins un élément.

- Résultat :

```
Minimum = 5
```

Rechercher le maximum

- Le principe est identique à celui du minimum, mais la comparaison est inversée.

```
int T[6] = {12, 5, 18, 7, 10, 8};
int i, max = T[0];
for (i = 1; i < 6; i++) {
    if (T[i] > max) {
        max = T[i];
    }
}
printf("Maximum = %d\n", max);
```

- Résultat :

```
Maximum = 18
```

- A retenir :
  - Pour le minimum, on utilise **`T[i] < min`**.
  - Pour le maximum, on utilise **`T[i] > max`**.

Compter les valeurs paires

- On souhaite compter le nombre d'éléments pairs.
- **Programme**

```
int T[6] = {12, 5, 18, 7, 10, 8};
int i, compteur = 0;
for (i = 0; i < 6; i++) {
    if (T[i] % 2 == 0) {
        compteur++;
    }
}
printf("Nombre de valeurs paires = %d\n", compteur);
```

- Résultat :

```
Nombre de valeurs paires = 4
```

- La variable **`compteur`** est initialisée à zéro et augmente chaque fois que la condition est vérifiée.

Rechercher une valeur

- On souhaite vérifier si une valeur **x** existe dans un tableau et déterminer l'indice de sa première occurrence.
- **Programme**

```
int T[6] = {12, 5, 18, 7, 10, 8};
int i, pos = -1, x = 7;
for (i = 0; i < 6; i++) {
    if (T[i] == x) {
        pos = i;
        break;
    }
}
if (pos != -1)
    printf("Valeur trouvee a l'indice %d\n", pos);
else
    printf("Valeur non trouvee\n");
```

Rechercher une valeur

- Résultat :

```
Valeur trouvee a l'indice 3
```

- La valeur **-1** indique ici que la valeur n'a pas été trouvée.
- L'instruction **break** arrête la boucle dès que la première occurrence est trouvée.

Compter le nombre d'occurrences d'une valeur

- Contrairement à la recherche précédente, on souhaite ici compter toutes les occurrences d'une valeur x.

```
int T[7] = {2, 5, 2, 8, 2, 4, 5};
int i, compteur = 0, x = 2;
for (i = 0; i < 7; i++) {
    if (T[i] == x) {
        compteur++;
    }
}
printf("Nombre d'occurrences = %d\n", compteur);
```

- Résultat :

```
Nombre d'occurrences = 3
```

- Il ne faut pas utiliser **`break`** dans cette boucle, car il faut examiner tous les éléments.

Inverser l'ordre des éléments

- On souhaite inverser les éléments du tableau sans utiliser un deuxième tableau.
- Tableau initial :

  `12  5  18  7  10  8`
- Tableau final :

  `8  10  7  18  5  12`

- **Principe**
  - Echanger le premier élément avec le dernier.
  - Echanger le deuxième élément avec l'avant-dernier.
  - Continuer jusqu'au milieu du tableau.

Inverser l'ordre des éléments

- **Programme**

```
#define N 6
...
int T[N] = {12, 5, 18, 7, 10, 8};
int temp, i;
for (i = 0; i < N / 2; i++) {
    temp = T[i];
    T[i] = T[N-1-i];
    T[N-1-i] = temp;
}
```

- La variable temporaire **`temp`** permet de conserver une valeur pendant l'échange.

Les tableaux à deux dimensions : les matrices

Pourquoi utiliser une matrice ?

- Supposons que l'on souhaite enregistrer les notes de cinq étudiants dans trois matières :
  - Algorithmique et Structures de Données (ASD)
  - Programmation C (C)
  - Mathématiques (Maths)
- Les données peuvent être organisées sous forme de tableau :

Etudiant	ASD	C	Maths
Etudiant 1	14	12	15
Etudiant 2	10	16	11
Etudiant 3	13	17	14
Etudiant 4	9	11	12
Etudiant 5	15	14	16

- Pour organiser ces données, on utilise un tableau à deux dimensions, appelé **matrice**.

Déclaration d'une matrice

- **Syntaxe générale**

```
type nom_tableau[nb_lignes][nb_colonnes];
```

- **Exemples**

```
int M[3][4];
float notes[5][3];
double A[10][10];
```

- **`M[3][4]`** possède trois lignes et quatre colonnes.
- **`notes[5][3]`** permet de stocker les notes de cinq étudiants dans trois matières.
- **`A[10][10]`** est une matrice carrée de dix lignes et dix colonnes.
- Une matrice de **`L`** lignes et **`C`** colonnes contient **`L * C`** éléments.

Représentation d'une matrice

- Considérons la matrice suivante :

```
int M[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
};
```

- Représentation :



- Chaque élément est identifié par deux indices :
  - Le premier indique la ligne.
  - Le deuxième indique la colonne.
- Par exemple, **`M[1][2]`** contient la valeur 6.

Initialisation d'une matrice

- On peut initialiser une matrice en indiquant les valeurs ligne par ligne.

```
int M[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

La première ligne contient **1, 2, 3.**
La deuxième ligne contient **4, 5, 6.**
La troisième ligne contient **7, 8, 9.**

- On peut également écrire :

```
int M[3][3] = {
    1, 2, 3,
    4, 5, 6,
    7, 8, 9
};
```

Accéder aux éléments d'une matrice

- Pour accéder à un élément, on utilise deux indices :

  `M[indice_ligne][indice_colonne]`

  - Le premier indice indique la ligne.
  - Le deuxième indice indique la colonne.
  - Les indices commencent à zéro.

- Exemple :

```
int M[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
};
printf("%d\n", M[0][0]);     // 1
printf("%d\n", M[2][1]);     // 8
```

A retenir : M[1][2] désigne l'élément situé à la deuxième ligne et à la troisième colonne.

Parcourir une matrice

- Pour parcourir tous les éléments d'une matrice, on utilise généralement deux boucles imbriquées:
  - La boucle extérieure parcourt les lignes.
  - La boucle intérieure parcourt les colonnes de chaque ligne.
- Exemple :

```
int M[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
int i, j;
for (i = 0; i < 3; i++){
    for (j = 0; j < 3; j++)
        printf("%d\t", M[i][j]);
    printf("\n");
}
```

Résultat :

```
1    2    3
4    5    6
7    8    9
```

Lire une matrice au clavier

- On souhaite lire les éléments d'une matrice de deux lignes et trois colonnes.

```
#include <stdio.h>

#define L 2
#define C 3

void main() {
    int M[L][C];
    int i, j;

    for (i = 0; i < L; i++) {
        for (j = 0; j < C; j++) {
            printf("M[%d][%d] = ", i, j);
            scanf("%d", &M[i][j]);
        }
    }
```

Lire une matrice au clavier

```
    printf("Matrice saisie :\n");

    for (i = 0; i < L; i++) {
        for (j = 0; j < C; j++) {
            printf("%d\t", M[i][j]);
        }
        printf("\n");
    }
}
```

- Les deux premières boucles permettent de saisir les éléments.
- **`scanf("%d", &M[i][j])`** stocke la valeur saisie dans la case correspondante.
- Les deux dernières boucles affichent la matrice ligne par ligne.

Traitements classiques sur les matrices

Calculer la somme des éléments d'une matrice

- On souhaite calculer la somme de tous les éléments d'une matrice.
- **Principe**
  - Initialiser une variable somme à zéro.
  - Parcourir toutes les lignes.
  - Parcourir toutes les colonnes.
  - Ajouter chaque élément à la somme.

Calculer la somme des éléments d'une matrice

```
int M[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
int i, j, somme = 0;

for (i = 0; i < 2; i++) {
    for (j = 0; j < 3; j++) {
        somme += M[i][j];
    }
}

printf("Somme = %d\n", somme);
```

Résultat :

```
Somme = 21
```

Calculer la somme de chaque ligne

```
int M[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
int i, j, somme;

for (i = 0; i < 2; i++) {
    somme = 0;

    for (j = 0; j < 3; j++) {
        somme += M[i][j];
    }

    printf("Somme ligne %d = %d\n", i, somme);
}
```

Résultat :

```
Somme ligne 0 = 6
Somme ligne 1 = 15
```

La diagonale principale d'une matrice carrée

- Dans une matrice carrée, la diagonale principale est constituée des éléments dont les deux indices sont égaux.
- Exemple :

$$M = \begin{pmatrix} 2 & 4 & 6 \\ 1 & 5 & 7 \\ 3 & 8 & 9 \end{pmatrix}$$

- Les éléments de la diagonale principale sont : M[0][0], M[1][1], M[2][2]

```
int i, somme = 0;
for (i = 0; i < 3; i++) {
   somme += M[i][i];
}
printf("Somme diagonale = %d\n", somme);
```

Addition de deux matrices

- Deux matrices de mêmes dimensions peuvent être additionnées élément par élément.
- Si A et B sont deux matrices, leur somme S est définie par :

$$S[i][j] = A[i][j] + B[i][j]$$

- Exemple :

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$$

- La matrice résultat est :

$$S = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

- Chaque élément de la matrice résultat est obtenu en additionnant les éléments correspondants des deux matrices.

Addition de deux matrices

```
#include <stdio.h>
#define N 2
void main() {
    int A[N][N] = {{1, 2}, {3, 4}};
    int B[N][N] = {{5, 6}, {7, 8}};
    int S[N][N], i, j;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            S[i][j] = A[i][j] + B[i][j];
    printf("Matrice somme :\n");
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            printf("%d\t", S[i][j]);
        }
        printf("\n");
    }
}
```

Résultat :

```
6    8
10   12
```